Log Aggregation

every log line in one searchable place

Watch the 4-min video

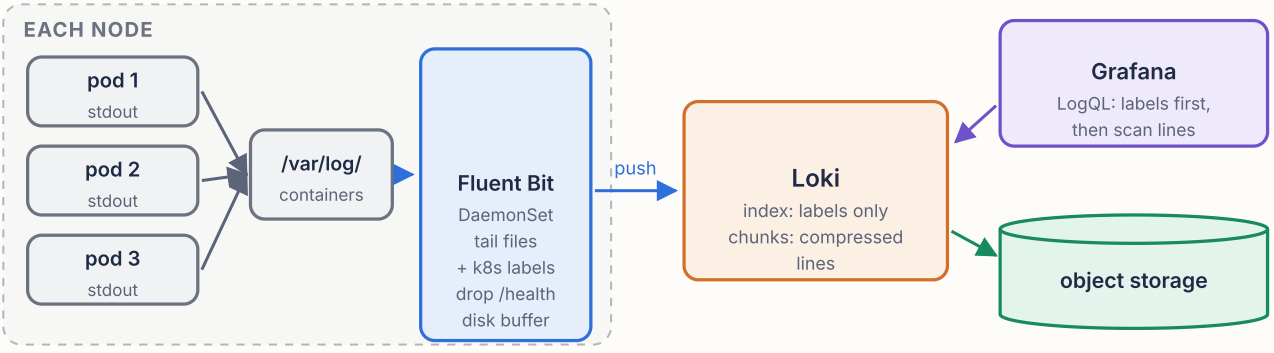
In one line: a shipper on every node collects, enriches, filters and buffers logs and sends them to a central store; choose how that store indexes (full text vs labels), cut volume at the source and tier retention, because at scale logs cost more than you expect.

1 Pods die, and their logs die with them

Fifty pods each write logs to their own node's disk. A pod crashes, Kubernetes replaces it, and its logs vanish. During an incident you're running `kubectl logs` pod by pod. Log aggregation puts every line, from every pod, in one searchable place that outlives the pods.

2 The pipeline: ship, store, search

Apps write structured JSON to **stdout** (lesson 60); the container runtime writes it to files on the node. A **shipper** runs on every node (a Kubernetes DaemonSet): **Fluent Bit**, Vector, Logstash or the OpenTelemetry Collector. It tails the files, parses, adds metadata (namespace, pod, container), filters noise and forwards to a central **store** that indexes for **search**.




```
{namespace="shop", service="orders"} |= "4bf92f35" # label selector, then line filter
```

Figure 1. Kubernetes logs to Loki: Fluent Bit tails container logs, enriches and filters them, buffers to disk and pushes to Loki, which keeps chunks in object storage.

3 The big choice: how the store indexes

Elasticsearch (ELK): full-text index
every word of every line is indexed
fast arbitrary search, aggregations
index can rival the raw data in size; RAM-heavy

Grafana Loki: label index
only a few labels indexed (namespace, app)
lines compressed in cheap object storage
filter by labels, then brute-force scan

Figure 2. Full-text indexing pays at write time for fast search; label indexing keeps writes and storage cheap and pays at query time.

	Elasticsearch / OpenSearch	Grafana Loki	Managed (Splunk, CloudWatch, Azure Monitor)
Search	any word, fast	fast if labels narrow it first	varies
Cost driver	index size, RAM, SSD	bytes scanned per query	GB ingested and retained
Ops work	high (shards, mappings, cluster)	moderate	low, more money
Best for	heavy ad-hoc search, analytics	Kubernetes logs, search by service + trace ID	teams buying time

In Loki, labels must be low-cardinality (namespace, app, level). A trace ID or user ID as a label creates a stream per value and breaks it; search for those with a line filter instead.

4 Controlling cost

Logging cost grows quietly: one debug line in a hot loop can cost more than the service. **At the source:** INFO in production, one rich line per request, sample repetitive successes. **In the shipper:** drop health-check and readiness-probe lines, throttle floods. **Find your ten noisiest sources** (by bytes per day) and fix those first. Then **tier retention**:

Tier	Typical age	Storage	Example streams
Hot	last few days	fast, indexed	everything you debug incidents with
Warm	weeks	cheaper nodes or object storage	app INFO and ERROR
Archive	months to years	object storage, cold classes	audit logs for compliance (lesson 55)

Set retention per stream: debug for days, audit for a year or more. Old logs are rarely read but always billed.

A ROBUST PIPELINE

- **Buffer to disk** in the shipper (Fluent Bit `storage.type filesystem`), so a slow store doesn't drop logs or block apps.
- **JSON logs**, so fields need no fragile regex parsing.
- **Redact** secrets and personal data before logs leave the node.
- **Monitor the pipeline itself:** alert on shipper retries, dropped records and ingestion lag.

WORKED EXAMPLE: KUBERNETES LOGS TO LOKI

Fluent Bit runs as a DaemonSet and tails `/var/log/containers/*.log`. Its **kubernetes** filter adds namespace, pod and container; a **grep** filter drops `/health` lines; a **throttle** filter caps repetitive floods. It pushes to Loki, which stores compressed chunks in object storage with retention of **14 days**. In Grafana, `{namespace="shop", service="orders"} |= "4bf92f35"` narrows by labels, then finds the trace ID (lesson 65), at a fraction of the cost of a full-text index.

5 Key terms

Shipper: per-node agent that collects and forwards logs

DaemonSet: Kubernetes object running one pod per node

Full-text index: index on every term of every line (Elasticsearch)

Label index: index on a few labels only (Loki)

LogQL: Loki query language: selector, then filters

Retention tier: hot, warm or archive storage by age

6 Interview questions

Q1. What does a log shipper do? BASIC

It runs on every node, collects logs, parses, enriches and filters them, buffers to disk and forwards them to central storage.

Q2. How do you control logging costs at scale? BASIC

Cut volume at the source (levels, sampling), drop noise like health checks in the shipper, use label-indexed storage such as Loki for most logs, and tier retention per stream.

Q3. ELK or Loki for a 200-service Kubernetes platform? SCENARIO

Loki if most searches start from service, namespace and a trace ID and cost matters; Elasticsearch if you need fast free-text search and analytics across all fields.

Q4. The log store is down for 20 minutes. What happens to logs? SCENARIO

With disk buffering, shippers queue and retry, then catch up; without it, they drop records or exert back-pressure on apps.

Q5. Why not make trace_id a Loki label? TRICKY

Each label value creates a separate stream; an unbounded label explodes the index and chunk count. Use a line filter or structured metadata instead.

Q6. Why do apps write to stdout instead of shipping logs themselves? TRICKY

It decouples the app from the logging backend: the platform can change shippers or stores, and a crashing app never blocks on a remote log server.

Next: SD-N-009 · Dashboards and Alerting backendroadmap.com